Implementing Raft for Distributed IoT Networks

Elaine Chao
A16894921

Abijit Jayachandran
A16794583

Pranav Mehta
A17323782

I. Introduction

Modern critical infrastructure systems increasingly rely on networks of IoT edge devices to control core functionalities. State inconsistencies across these devices, which are often caused by factors such as node crashes, power loss, or network issues like latency and wireless packet loss, can lead to unintended behavior, resulting in potentially catastrophic consequences. System-wide Replicated State Machines (RSMs) mitigate these risks by ensuring a deterministic system state, providing high fault tolerance and availability. The foundation of an RSM is consensus and total ordering, requiring nodes to agree on both the operations performed and their execution sequence. Raft is a consensus protocol that fulfills these requirements, as it matches the performance of the well-known Paxos protocol but is significantly simpler, making it an ideal candidate for distributed IoT networks. Our project explores building fault-tolerant, consistent distributed IoT networks using a Rust implementation of Raft. Because these networks typically consist of low-power embedded devices with limited resources, we will also implement log compaction to minimize the memory and storage footprint of our implementation.

II. Existing Work & Dependencies

The Raft consensus algorithm provides the theoretical foundation for our implementation [1]. Originally developed to serve as a more understandable alternative to Paxos, Raft decomposes the overarching consensus problem into distinct, independent subproblems: leader election, log replication, and safety. A core aspect of the protocol is the concept of strong leadership, where log entries only flow in a single direction, from the leader to the other servers. During standard operation, the cluster contains exactly one leader, while all other servers act as passive followers. The leader has sole responsibility for accepting client commands, appending the commands to its local write-ahead log, and issuing `AppendEntries` remote procedure calls (RPCs) in parallel to replicate the entries across the cluster. To handle leader failures, Raft employs a heartbeat mechanism coupled with randomized election timeouts, which ensures that split votes are rare and resolved rapidly without complex mechanisms.

To guarantee that all state machines execute the exact same sequence of commands without inconsistency, the Raft protocol strictly enforces five invariant safety properties:

- **Election Safety:** At most one leader can be elected in a given term.
- **Leader Append-Only:** A leader never overwrites or deletes entries in its log, instead it only appends new entries.
- **Log Matching:** If two logs contain an entry with the same index and term, then the logs are identical in all entries up through the given index.
- **Leader Completeness:** If a log entry is committed in a given term, then that entry will be present in the logs of the leaders for all higher-numbered terms.
- **State Machine Safety:** If a server has applied a log entry at a given index to its state machine, no other server will ever apply a different log entry for the same index.

To bring this framework to fruition, our implementation leverages a highly concurrent Rust technology stack. Network communication and the essential RPCs required for leader elections and log replication are handled by the `tonic` gRPC transport layer, alongside `prost` for efficient Protocol Buffer serialization. The `tokio` crate provides the asynchronous runtime necessary to manage parallel network requests, variable latency, and Raft's randomized heartbeat timers. Finally, the `redb` embedded database supplies the high-performance, persistent storage mandated by Raft, ensuring that critical state variables and log entries are safely committed to stable storage before a server responds to any RPC.

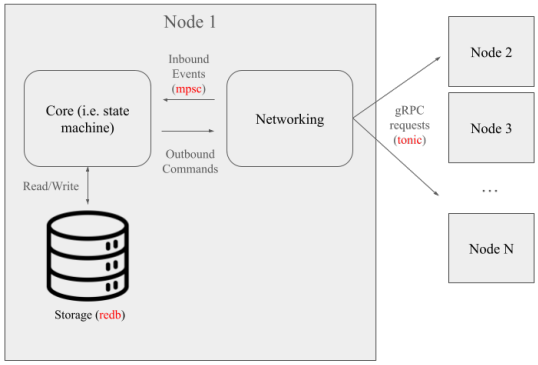


Fig. 1: System design of our Raft implementation.

III. Implementation Details

The Raft implementation is composed of three primary components: the logging mechanism with inbuilt log compaction for memory efficiency, RPC calls and the communication network setup between our nodes, and the control logic that handles consensus and the invariant properties of Raft. We currently perform replication of a single boolean state. This can

fairly easily be scaled up as required by the application. Please see Fig. 1 for the overall system architecture. Now, we will go into the details of each individual component/layer. You can find our source code here: https://github.com/abijitj/RAFT.

A. Storage

We have implemented log compaction as part of the same persistent storage abstraction used for the WAL. The storage layer exposes an interface for appending entries, retrieving entries, and truncating the log, and compaction is realized by advancing a snapshot index and removing older entries once the committed log has moved past the configured threshold. On Unix-based systems, this is built on the Redb-backed storage implementation, so the same high-performance key-value store functionality underpins both ordinary logging and compaction. If this code were deployed on a real embedded system, the same compaction interface could be backed by NVS or an attached SD card without changing the Raft storage high level logic.

B. Network

For easier readability and testing, we decided to decouple the RPC call logic from the core consensus logic. This was done by creating two multiple producer, single consumer (mpsc) channels between the network layer and core. Whenever a request is received at the network layer this is formatted as a RaftEvent struct (in Rust) and sent to the core logic through the inbound channel. Similarly, when the core wants to send an outbound request/response, the network layer reads it from the outbound channel and makes the relevant RPC call.

By separating the network layer, we can independently test the network calls and also filter out commands from specific nodes as needed for testing. For example, in order to simulate a one-way network packet loss, we can block incoming requests from a specific node at the network itself. That way the core logic can naturally continue to behave as if that message never arrived.

Each outbound RPC command includes a one-shot response channel that associates the eventual network result with that particular call. Our `RequestVote` is the same as in the Raft paper. For `AppendEntries`, our core includes the request's preceding and final log indices when dispatching the RPC, and the asynchronous response-forwarding task includes those values when returning the result to the core event queue. This correlation is maintained locally and did not require adding fields to the Protocol Buffer RPC messages.

C. Core

The Raft core is implemented as a single asynchronous event loop that owns all consensus state for a node. Though network requests and timers execute concurrently, they are converted into `RaftEvent` values and processed sequentially by the core. RPC responses are returned to the same event queue. This design allows network operations to remain asynchronous while preventing concurrent modifications to role, term, log, and commit state without requiring locks around the consensus algorithm.

Each node maintains the follower, candidate, and leader roles together with Raft's persistent and volatile state. The persistent state consists of `currentTerm`, `votedFor`, and the log. Term and vote changes are written to stable storage before the corresponding operation completes. Persistent-storage failures cause the node to terminate rather than continue with potentially inconsistent state. Volatile state includes `commitIndex`, `lastApplied`, the replicated boolean value, and the known leader. Leaders additionally maintain separate `nextIndex` and `matchIndex` values for every follower.

Followers use randomized election deadlines between 150 and 300 ms. The random-number generator is seeded using the node identifier and current term, producing different but deterministic timeouts under Shadow. Receiving a valid `AppendEntries` or `InstallSnapshot` request resets this deadline. When the deadline expires, the node increments its term, becomes a candidate, votes for itself, persists the new term and vote, and sends `RequestVote` requests containing the index and term of its latest logical log entry. A candidate becomes leader only after collecting votes from a strict majority of the configured nodes. Duplicate replies cannot increase the vote count because votes are stored in a set indexed by node identifier.

Any node that observes a higher term immediately persists that term, clears its previous vote, and returns to the follower state. A leader isolated by a network partition does not automatically become a candidate but instead remains leader in its local term and continues attempting replication. However, it cannot commit new entries without acknowledgments from a majority. Once communication is restored and it observes the higher term established by the majority partition, it steps down and accepts the current leader's log.

The voting logic provides the implementation-level checks supporting Election Safety and Leader Completeness. A node grants at most one vote per term because `votedFor` is persisted and cleared only after observing a higher term. It also rejects candidates whose logs are less up to date than its own. Log freshness is determined by comparing the term of the candidate's final entry, then by its index when the terms are equal. Consequently, an outdated candidate cannot win an election using votes from a majority containing nodes with more recent logs.

Upon becoming leader, a node initializes each follower's `nextIndex` to the position one past the end of its logical log and initializes `matchIndex` to zero. Every 50 ms, the leader sends a heartbeat, an `AppendEntries` request to each follower. It will also send an `AppendEntries` request to replicate data whenever a client command is accepted. Only the leader accepts commands. If the client attempts to talk to a follower, the follower rejects them and, when available, returns the known leader as a redirection hint. A successful client request is not acknowledged until its entry has been committed and applied.

Each AppendEntries request identifies the index and term immediately preceding the transmitted batch. A follower accepts the batch only if that position matches its local log or snapshot boundary. If an incoming entry conflicts with a stored entry at the same index, the follower invokes a dedicated suffix-truncation operation that deletes the conflicting entry and every entry following it while preserving the shared prefix. The leader's replacement entries are then persisted. Prefix deletion used for snapshot compaction and suffix deletion used for conflict repair are separate storage operations, preventing conflict recovery from accidentally removing already-agreed entries.

Because multiple AppendEntries calls to the same follower may be in flight concurrently, responses can arrive in a different order from their requests. The core therefore records the prevLogIndex and final transmitted log index for each individual call and carries these values into the corresponding response event. A successful response advances matchIndex only to the final index included in that specific request, rather than using replication state that may have changed while the RPC was in flight. The follower's nextIndex is then advanced monotonically so that a delayed success cannot move it backward.

Rejected calls are handled using the same request-specific metadata. The leader decreases nextIndex only if it equals the first index attempted by the rejected call. If a newer response has already changed the follower's replication position, the rejection is stale and is ignored. Otherwise, the leader backs up by one index and retries until it locates the last shared position. This response correlation prevents delayed successes from being mistaken as acknowledging newer entries and prevents delayed failures from undoing more recent replication progress. Together with the consistency checks and suffix-replacement rules, this allows stale followers to catch up while ensuring that matchIndex accurately represents entries the follower actually acknowledged.

The leader advances commitIndex by examining the replication indexes of all voting nodes. An index is committed only when it has been replicated to a strict majority and its entry belongs to the leader's current term. The latter restriction prevents a new leader from declaring an entry from an earlier term committed solely because that entry was subsequently copied to a majority. Once a current-term entry is committed, preceding entries become committed indirectly through the matching log prefix. This commit rule, together with the election freshness restriction, supplies the core support for Leader Completeness.

Committed entries are applied strictly in increasing index order. The core repeatedly advances lastApplied until it reaches commitIndex, updates the boolean state machine from each entry, and only then completes any waiting client request. Followers restrict their commit index to the smaller of the leader's announced commit index and the end of their own logical log. Since conflicting uncommitted suffixes are replaced before application and commands are applied only after commitment, this path supports State Machine Safety. Leaders never rewrite their own entries and only append client commands, maintaining the Leader Append-Only property.

Snapshot boundaries are treated as entries in the logical log even though their corresponding commands have been removed from ordinary log storage. The logical last index is computed as the greater of the final stored entry index and snapshotIndex. When no post-snapshot entries remain, the term at the logical final index is obtained from snapshotTerm. Consequently, compaction cannot reset the apparent end of the log to zero or cause newly appended entries to reuse indices that were already included in a snapshot.

When a follower's required nextIndex is no longer available because the leader has compacted that prefix, the leader sends an InstallSnapshot request instead of AppendEntries. The snapshot records the last included index and term together with the replicated state-machine value. A follower persists a newer snapshot before updating its snapshot boundary, commit index, applied index, and state-machine value. After successful installation, the leader sets the follower's replication indexes to the first position following the snapshot and resumes ordinary log replication. This allows compaction to reduce persistent storage while preserving the index and term information required by elections, log consistency checks, and subsequent appends.

IV. Experimental Setup & Evaluation

Our implementation runs within a standard Unix environment. Rather than deploying directly to memory-constrained embedded hardware immediately, we keep the system portable by targeting Linux processes that communicate over TCP. This approach allows us to use a network simulator for controlled, repeatable testing. While we initially considered the CORE simulator with the EMANE plugin to emulate resource-constrained wireless environments, CORE lacks strict deterministic execution, rendering it poorly suited for debugging distributed consensus issues.

We instead utilize the Shadow network simulator, which runs each node as a real Linux process under a deterministic, discrete-event virtual clock [2]. This allows us to perfectly replicate complex state transitions without relying on timing luck. Every Shadow test is configured with a cluster.json describing the node-to-IP mapping and a YAML file specifying the network topology, per-node start and shutdown times, and injected write commands. This grants us absolute control over simulating the unpredictable failure scenarios typical of embedded environments.

Our test suite is organized around the five invariant properties of Raft, rigorously evaluating them against the failure modes most relevant to embedded deployments: unexpected node crashes (simulating battery exhaustion, out-of-memory crashes and other node-specific issues), persistent storage failures, network partitions, and slow or lossy network links. We will now describe a subset of the most important tests that we ran on our design.

Correctness Tests

a) Three Happy Nodes: As a baseline sanity check, three nodes on a 1 Gbit switch with 10 ms latency run for 10 simulated seconds. We verify that a leader is elected and heartbeats proceed normally to prevent unnecessary elections before adding any fault injection.

b) Leader Replication: Three nodes run on the same switch, with writes injected on multiple nodes simultaneously at overlapping millisecond offsets. This tests that only the elected leader appends and commits entries to its log, and that conflicting client write attempts directed at followers are correctly redirected, preserving the Leader Append-Only property.

c) Leader Crash: In a three-node cluster, the leader is killed with a SIGTERM at $t = 5$s. The remaining two nodes must successfully detect the loss of `AppendEntries` heartbeats, allow their randomized election timeouts to elapse, hold a new election, and continue operating. Success requires observing a clean term transition and the establishment of a new, stable leader.

d) Leader Crash Before Commit: In a five-node cluster, the leader receives an injected write at $t = 3$s and is killed at $t = 5$s before it can issue the commit. The remaining four nodes must either safely commit the entry (if it successfully replicates to a majority before the crash) or safely discard it. This directly tests the Log Matching and Leader Completeness invariants, ensuring the cluster correctly handles the restriction on committing entries from previous terms, as detailed in Section 5.4 of the Raft paper.

e) Retrying AppendEntries: In a three-node setup, node 2 experiences a simulated network block from $t = 2002$ms to $t = 2500$ms while a write is in flight. The leader must continuously retry the `AppendEntries` RPC and decrement its `nextIndex` until the follower becomes reachable, the logs converge, and the entry is committed. This verifies the robust consistency check and backtracking logic inherent to Raft's replication phase.

f) Leader Completeness Across Multiple Terms: A five-node cluster first commits a `true` entry before several nodes are killed and restarted at staggered times, forcing the cluster to conduct elections across multiple terms. Once a majority becomes available again, a later leader appends a `false` entry. The test verifies that every node's final WAL contains the original committed entry at index 1 as well as the later entry, and that all logs converge. This directly evaluates Leader Completeness by ensuring that an entry committed in an earlier term remains present under every subsequently elected leader.

g) Crash During Conflict Repair: Node 3 is isolated after becoming leader and appends an uncommitted `true` entry. Nodes 1 and 2 subsequently elect a higher-term leader and append `false` at the same log index. After the partition heals, node 3 crashes immediately after it persistently truncates its conflicting suffix but before it appends the replacement entry. Node 3 is then restarted from the partially repaired WAL and must complete synchronization with the leader. The test verifies that the replacement entry is installed on all nodes and that the discarded entry does not reappear after recovery.

h) Interrupted Snapshot Installation: Node 1 is taken offline while nodes 2 and 3 commit enough entries to cross a reduced compaction threshold and create a snapshot. When node 1 first rejoins, it is terminated immediately before persisting the incoming `InstallSnapshot` request, simulating a crash or power failure during snapshot recovery. After a second restart, the leader retries the snapshot transfer, after which node 1 has to persist the snapshot and accept a normal post-snapshot entry through `AppendEntries`. This test demonstrates that an interrupted snapshot transfer can be retried without corrupting the follower's persistent state or preventing subsequent replication.

i) Leader Crash After Majority Replication: The initial leader appends a `true` entry and crashes immediately after acknowledgments from a majority advance its local `commitIndex`, but before a later heartbeat can propagate the updated commit index to the followers. The remaining nodes elect a new leader, which appends a `false` entry in its current term. Committing this new entry needs indirectly commit and apply the preceding majority-replicated entry from the earlier term. The original leader later rejoins and catches up, and the test verifies that every final WAL contains both entries in the same order. This test ensures that a majority-replicated entry is preserved across leadership changes even when commitment notification is interrupted.

Partition Tests

j) Node Partitioned: A five-node cluster is partitioned from with node 1 isolated on one network segment and nodes 2 through 5 connected on another. Because a majority requires three votes, the isolated node can repeatedly begin elections and increase its local term but never manages to become leader or commit an entry. The four connected nodes have a quorum and so can elect a leader and successfully commit a write.

k) Minority Partition: A five-node cluster is split into a majority partition (nodes 1–3) and a minority partition (nodes 4–5), with a 100% packet loss rate on all cross-partition edges. Nodes 1–3 successfully elect a leader and commit a write injected at $t = 10$s. Meanwhile, nodes 4–5 continually time out and increment their terms but cannot reach a quorum, preventing them from committing. Our invariant checker confirms that no entry is erroneously committed on the minority side, verifying Election Safety.

l) Leader Partitioned: In a five-node cluster, node 3 elected is as leader, and then isolated on one network segment at $t = 2$s. In the absence of the previous leader, nodes 1, 2, 4 and 5 form a majority, elect a leader, and safely commit a write injected at $t = 4$s. Meanwhile, node 1 continues sending `RequestVotes` with higher and higher term numbers. It also still believes itself to be leader, and will accept client writes and append them to its log, but is unable to commit without a majority, as shown by the `commitIndex` only increasing for the majority partition. Then node 1 rejoins the majority, starting another election, but is not elected leader.

The new leader performs conflict resolution on node 1's logs and the divergent entries are deleted. This verifies both that a partitioned, stale node with a potentially higher term number cannot be elected leader because its log is not up to date, and that it is safe for an isolated node to still believe it is the leader until told otherwise.

m) Even Split: Six nodes are split exactly 3–3 across two disconnected networks, with writes simultaneously attempted on both halves at $t = 10$s. Because neither side can reach the required majority of 4, no leader is elected and no entry is committed. This confirms that the system prioritizes safety over liveness and correctly stalls rather than permitting a split-brain scenario.

n) Asymmetric Partition: A three-node cluster experiences a one-way network failure in which nodes 1 and 2 reject messages sent by the initial leader, node 3, while node 3 can still receive their RPCs. Nodes 1 and 2 therefore elect a higher-term leader and commit a new entry despite the previous leader continuing to send replication requests. When communication is restored, node 3 observes the higher term, steps down, and synchronizes its log with the new leader. The test checks that the one-way block was exercised, that the stale leader recognized the higher term, and that all three WALs ultimately converge.

Node Churn and Availability

o) Node Churn: Modeled after the battery-exhaustion failure pattern common in embedded deployments, five nodes operate on a shared 10 ms switch. Nodes 3, 4, and 5 are killed sequentially at $t = 5$s, $t = 10$s, and $t = 15$s respectively, while writes are injected on nodes 1, 2, and 3 at staggered times. Because the cluster never drops below a functional majority of two surviving nodes (nodes 1 and 2), writes continue to be committed successfully throughout the degradation.

p) 100 Nodes Simple: A 100-node cluster operates on a 1 Gbit switch for 10 simulated minutes. Seven nodes are killed at staggered intervals throughout the run, remaining well below a majority failure. This verifies that randomized election timeouts and log replication continue to scale and converge efficiently even with intermittent minority failures in a densely populated network.

q) 100 Nodes Complex: The same 100-node cluster faces a highly aggressive failure pattern: 16 nodes are killed in a sudden burst at $t = 10$s, another 10 at $t = 60$s, and one more at $t = 120$s, across a 420-second simulation. This stresses the system by ensuring the leader can continue replicating through multiple, overlapping waves of node churn while accurately maintaining the `matchIndex` array for a massive follower pool.

Unreliable Network

r) Unreliable Network: Five nodes operate on a switch with a 50 ms base latency and an aggressive 40 ms jitter. Because Shadow does not natively support a direct packet drop parameter, this high jitter is utilized to force RPC timeouts, perfectly mimicking dropped packets in a lossy wireless environment. A write is injected at $t = 10$s, and the cluster runs for 60 simulated seconds. Success requires the write to be durably committed despite the heavily degraded link conditions, verifying Raft's tolerance to non-Byzantine timing issues.

Persistent Storage

s) Persistent Metadata Restart: A single-node cluster is killed abruptly with SIGKILL at $t = 2$s and restarted on the same host at $t = 3$s. The restarted process must correctly restore its persisted `currentTerm` and `votedFor` fields from the `redb` write-ahead log to begin its next election in the proper term. This tests the strict durability guarantee required by Raft to prevent a server from voting twice in the same term.

t) Storage Failure Fail-Stop: A dedicated `storage_failure_probe` binary runs against an injected, corrupted write-ahead log. The test verifies two critical safety behaviors: a metadata read failure upon startup prevents the node from constructing its Raft state entirely, and a metadata write failure during normal operation forces the Raft core to immediately terminate. The system must fail-stop rather than operate on inconsistent state.

Log Compaction

u) Log Compaction: A three-node cluster is designed to exercise the full snapshot and catch-up mechanism vital for memory-constrained devices. Node 1 is killed at $t = 3$s. Node 3 wins the subsequent election and issues 50 sequential writes, crossing the compaction threshold and triggering a snapshot at $t = 56$s on nodes 2 and 3, discarding the superseded log entries. Node 1 is restarted at $t = 59$s. Because Node 1's `nextIndex` lags behind the leader's `lastIncludedIndex`, the leader accurately falls back to sending an `InstallSnapshot` RPC. Node 1 installs the snapshot and smoothly transitions back to receiving normal `AppendEntries` RPCs for subsequent writes.

Chaos

v) Chaos: Designed as the ultimate stress test, a seven-node cluster operates with 25 ms base latency and 25 ms jitter. Multiple nodes issue competing writes at interleaved intervals ($t = 10$s, $t = 30$s, $t = 50$s, and $t = 70$s), while an overlapping sequence of node failures occurs at $t = 20$s, $t = 25$s, $t = 40$s, and $t = 60$s. The cluster carefully maintains a surviving majority at all times. After 120 simulated seconds, the invariant checker verifies that all surviving nodes have converged to an identical state, proving that State Machine Safety is upheld despite concurrent failures, conflicting writes, and severe network degradation. We also ran a slightly more intense version of this test (Chaos V2) which performs 99 writes at various time intervals when Node 6 is the leader and does the same when Node 7 is the leader. The afore-mentioned tests hopefully convince the reader of the correctness of this Raft implementation.

V. Conclusion

Our project provides a complete Rust implementation of Raft. This implementation preserves Raft's safety invariants through persistent term and vote state, majority replication, log freshness checking and conflict repair. Across our simulated scenarios, the invariant checker found no violations and nodes consistently reached agreement with each other after communication was restored following crashes or network failures. These results demonstrate the correctness of our implementation under the fail-stop model.

Our design provides a foundation suitable for distributed IoT and embedded applications. It is memory safe without needing garbage collection, and separating the network, core and storage layers means that it is open for modification to adapt to platform-specific transports and storage. It is also lightweight. Log compaction limits long term storage growth and snapshotting allows devices to catch up without having to replay the entire log. The storage, election and replication performance measurements show that our protocol can maintain consensus with modest overhead under node churn and network disruption.

VI. Future Work

The next steps for this Raft implementation would be to optimize it for embedded applications and implement automatic cluster membership changes. An approach for the latter has already been described in the original Raft paper [1]. For the former, there are several ways we can optimize the design for certain applications depending on our goals.

If the embedded application is in an environment with lower network bandwidth, we can tweak design to reduce network usage. For example, instead of having a steady heartbeat every 50ms, we could have an adaptive one that decreases the heartbeat frequency during periods of stability and increases it when an anomaly is detected. If we are in an environment with high network volatility (e.g. one node could disconnect and rejoin with a much higher term number, leading to a re-election of the quorum), we could design an extra pre-voting stage which checks if a new node could actually be a leader before incrementing everyone's term number.

If the embedded system has strict memory restrictions, we can also modify how the logging and persistent storage is managed. For example, we can replace the in-memory dynamic log with a fixed size circular buffer so that we never pressure the RAM. This has the drawback of possibly having to read from slower storage much more often.

Additionally, it would be beneficially to setup a framework designed for performance testing. The tool that we used is great for deterministic, repeatable testing when aiming to rigorously check correctness [2]. However, it falls short when used for profiling. Shadow is a discrete event network simulator and as a result is not ideal for performance profiling since it fundamentally breaks does each event at the OS-level. A better alternative for profiling may be network emulators like mininet or CORE.

Thus, there are many next steps one can take to optimize this Raft design for embedded applications. But we hope that a foundation has been laid such that future modifications do not require major systemic changes.

References

[1] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, USENIX ATC'14, pages 305–320, Berkeley, CA, USA, 2014. USENIX Association.

[2] R. Jansen, J. Newsome, and R. Wails. Co-opting Linux Processes for High-Performance Network Simulation. In *USENIX Annual Technical Conference*, 2022.